

Intro to Online Optimization and Learning

CS245: Online Optimization and Learning

Xin Liu
SIST, ShanghaiTech University

General information about this course

General information:

- 12 weeks and 3 credits
- “Prerequisites”: Algorithms and data structures, Probability and statistics, and Convex optimization

Grades:

- Three problem sets (30%) : algorithm implementation (with python) and analysis (with latex).
- Final project (60%) : report (45%) and presentation (15%). Group (≤ 3) and indicate your individual contribution.
- Participation and quiz: 10%.

Late homework policy:

- 3 late days (75% $\rightarrow$ 50% $\rightarrow$ 25%) for homework and no late days for the final project!

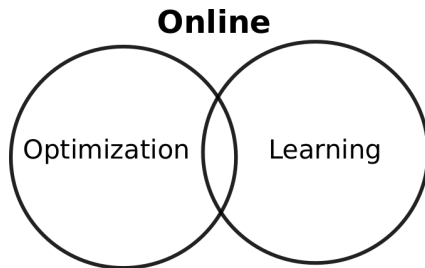
General information about this course

TA: Hengquan Guo (guohq@shanghaitech.edu.cn), 2-209, SIST

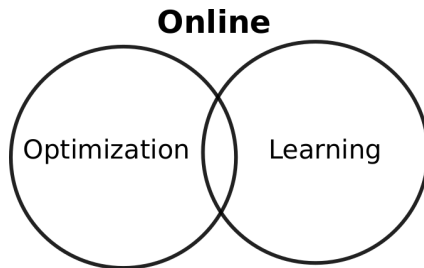
Office hour: 10:00–11:00am on Friday at 2-302H, SIST

Academic Integrity!!!

- ShanghaiTech academic integrity policy.
- Problem sets: be independent, feel free to discuss, and cite when you borrow ideas from books, internet, and peers, etc.
- Final project: indicate your individual contribution in the report.

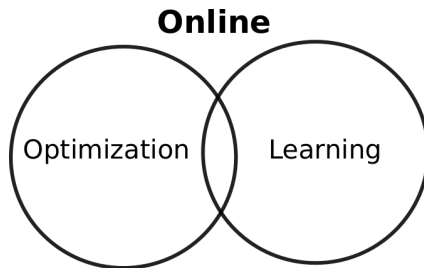


What does “online” mean?



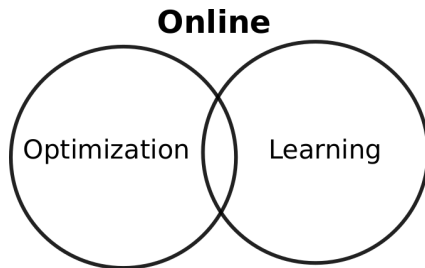
What does “online” mean?

- Antonym of offline/batch



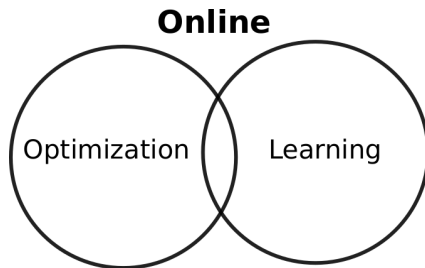
What does “online” mean?

- Antonym of offline/batch
- Stochastic and uncertainty



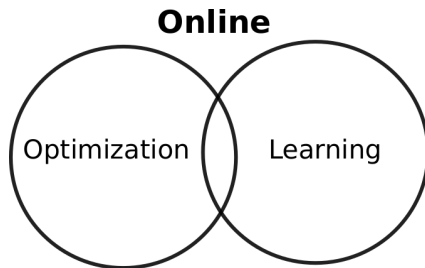
What does “online” mean?

- Antonym of offline/batch
- Stochastic and uncertainty
- Adaptive, efficient, and robust



What does “online” mean?

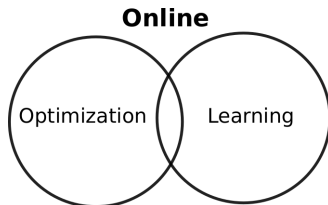
- Antonym of offline/batch
- Stochastic and uncertainty
- Adaptive, efficient, and robust
- Challenging and powerful



What does “online” mean?

- Antonym of offline/batch
- Stochastic and uncertainty
- Adaptive, efficient, and robust
- Challenging and powerful

Online decision-making
under uncertainty



Applications

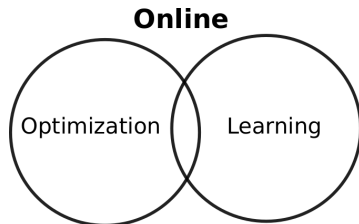
- Recommendation systems
- AI gaming (Alpha Go)
- Healthcare (clinic trial)
- Wireless nets (resource allocation)
- LLMs (finetuning/reasoning)
- ... Your Research ...

Theory

- Online convex optimization
- Stochastic/Adversarial Bandits
- Reinforcement learning

Algorithms

- Online gradient descent
- Upper confidence bound



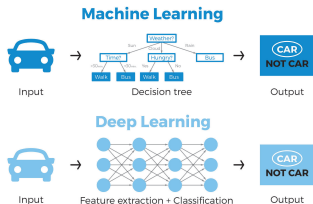
The style of the courses:

- Motivated examples
- Problem formulation via online O&L
- Intuition and online algorithms
- Performance analysis (e.g., regret)
- Discussions (e.g., optimality)

Offline learning

We know (offline or batch) learning:

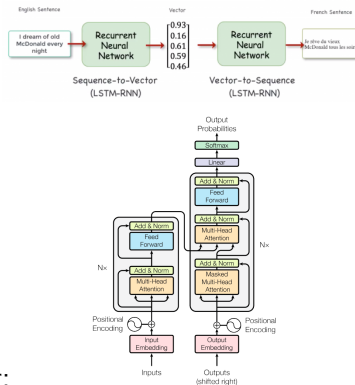
- Objective recognition
- Machine translation



Key elements in (offline or batch) learning:

- Data
- Models and algorithms

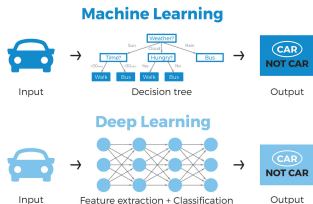
Encoder-Decoder Architecture



Offline learning

We know (offline or batch) learning:

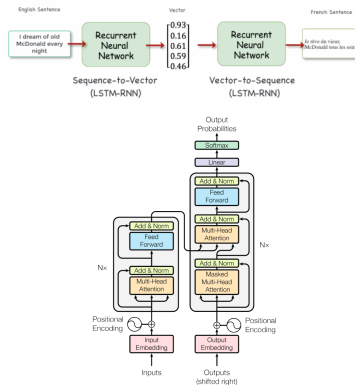
- Objective recognition
- Machine translation



(Offline or batch) learning:

- Data is important, which is given and fixed (ImageNet, SQuAD).
- Tune models or algorithms (linear or non-linear, structure of deep nets).

Encoder-Decoder Architecture



Offline Optimization

We know (offline) optimization:

$$\begin{array}{ll}\min_{x \in \mathcal{X}} & f(x) \\ \text{s.t.} & g(x) \leq 0\end{array}$$

How to solve an optimization problem?

We know (offline) optimization:

$$\begin{array}{ll}\min_{x \in \mathcal{X}} & f(x) \\ \text{s.t.} & g(x) \leq 0\end{array}$$

How to solve an optimization problem?

- We solve it with solvers (e.g., CVX, Gurobi)

We know (offline) optimization:

$$\begin{array}{ll}\min_{x \in \mathcal{X}} & f(x) \\ \text{s.t.} & g(x) \leq 0\end{array}$$

How to solve an optimization problem?

- We solve it with solvers (e.g., CVX, Gurobi)
- We like “linear” and run simplex algorithm
- We like “convex” and check KKT condition

We know (offline) optimization:

$$\begin{array}{ll}\min_{x \in \mathcal{X}} & f(x) \\ \text{s.t.} & g(x) \leq 0\end{array}$$

How to solve an optimization problem?

- We solve it with solvers (e.g., CVX, Gurobi)
- We like “linear” and run simplex algorithm
- We like “convex” and check KKT condition
- Just run (stochastic) gradient descent
-

A well-known example of offline learning and optimization

Consider linear regression (LR) for:

- Shanghai Putong house price prediction

Given historical/batch data $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_T, y_T)\}$, we do LR

$$\min_{\mathbf{w}} \|\mathbf{X}^T \mathbf{w} - \mathbf{y}\|_2^2$$

Solve LR:

- in a close form?
- with (stochastic) gradient descent
- with regularizers always

Goal:

avoid overfitting
&
min the generalization error.

A well-known example of offline learning and optimization

Consider linear regression (LR) for:

- Shanghai Putong house price prediction

Given historical/batch data $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_T, y_T)\}$, we do LR

$$\min_{\mathbf{w}} \|\mathbf{X}^T \mathbf{w} - \mathbf{y}\|_2^2$$

Solve LR:

Goal:

- in a close form?
- with (stochastic) gradient descent
- with regularizers always

avoid overfitting
&
min the generalization error.

How about we gain data in an online way (with distribution drift)?

How about your model (policy) affects your data?

Offline learning is ending?

Ilya Sutskever: "Sequence to sequence learning with neural networks: what a decade"

Pre-training as we know it will end

Compute is growing:

- Better hardware
- Better algorithms
- Larger clusters

Data is not growing:

- We have but one internet
- **The fossil fuel of AI**

How about we **do not have any more data?**

"Offline training or pretraining is **ending?!"** Ilya at NeurIPS24.

From offline to online learning?

Ilya Sutskever: "Sequence to sequence learning with neural networks: what a decade"

What comes next?

- "Agents"??
- "Synthetic data"
- Inference time compute $\sim O$

Ilya Sutskever: "Sequence to sequence learning with neural networks: what a decade"

What comes next? The long term

Superintelligence

- Agentic
- Reasons
- Understands
- Is self aware

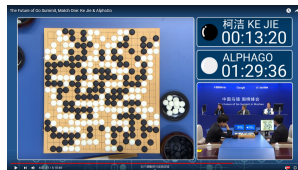
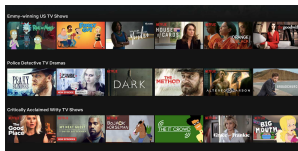
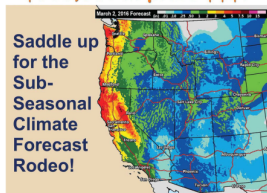
What comes next? **online, continual, and reinforcement learning?**

Motivated examples

Online optimization and learning:

- Movie recommendation
- Go game
- Climate prediction
- LLMs

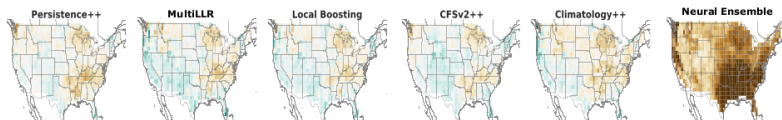
\$800,000 in prize \$\$\$!



Subseasonal climate forecast¹

Subseasonal Climate Forecast Rodeo by the Bureau of Reclamation and the National Oceanic and Atmospheric Administration:

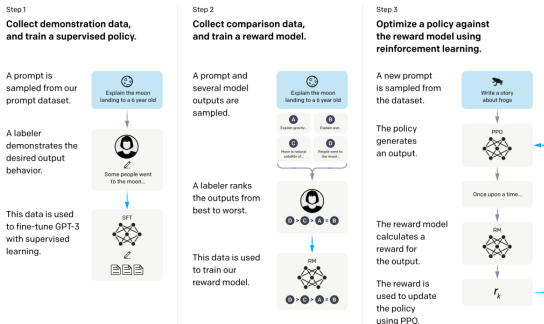
- Temperature and precipitation of 2 ~ 6 weeks in the Western US (Holy grail of forecasting).
- Diverse collection of forecasting models.
- At least one model performs well each year (but unclear which apriori).



Perfectly fit online optimization and learning?

¹Genevieve E Flaspohler, etc., “Online Learning with Optimism and Delay”. ICML, 2021.

LLM Finetuning



“Reinforcement learning from human feedback” for LLM finetuning²:

- Learn a base/ref policy from supervise learning
- Learn reward models via human/AI feedback
- Policy optimization based on reward models

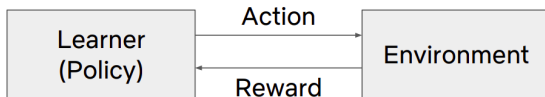
²Long Ouyang, etc., “Training language models to follow instructions with human feedback”. NeurIPS, 2022.

A first look at online learning

Online learning framework

For $t = 1, \dots, T$:

- **Learner:** Submit action/decision x_t .
 - **Environment:** Observe the feedback: loss $f_t(\cdot)$ or reward $r_t(\cdot)$.
-



A first look at online learning

Online learning framework

For $t = 1, \dots, T$:

- **Learner:** Submit action/decision x_t .
 - **Environment:** Observe the feedback: loss $f_t(\cdot)$ or reward $r_t(\cdot)$.
-

For different applications, $x_t, f_t(\cdot), r_t(\cdot)$ have different meaning:

- subseason climate prediction
- movies/items recommendation
- matching in crowdsourcing
- resource allocation in wireless system
- LLMs finetuning/reasoning
-

A first look at online learning

Online learning framework

For $t = 1, \dots, T$:

- **Learner:** Submit action/decision x_t .
 - **Environment:** Observe the feedback: loss $f_t(\cdot)$ or reward $r_t(\cdot)$.
-

The regret of a learner is defined to be:

$$\text{Regret}(T) = \sum_{t=1}^T f_t(x_t) - \min_{x \in \mathcal{X}} \sum_{t=1}^T f_t(x),$$

$$\text{Regret}(T) = \max_{x \in \mathcal{X}} \sum_{t=1}^T r_t(x) - \sum_{t=1}^T r_t(x_t).$$

Objective of learner: minimize the $\text{Regret}(T)$!

A first look at online learning: subseasonal climate forecast

Aggregating weather prediction:

Initialization: N experts/models.

For each time period $t = 1, \dots, T$:

- **Learner:** Combine predictions from N experts/models and make a prediction on the weather.
 - **Environment:** Observe the outcome.
-

Objective: find the best expert in hindsight and minimize the regret as follows

$$\text{Regret}(T) = \sum_{t=1}^T f_t(x_t) - \sum_{t=1}^T f_t(\text{best expert})$$

Prediction with Expert Advice

Aggregating weather prediction:

Initialization: N experts/models.

For each day $t = 1, \dots, T$:

- **Learner:** Combine predictions from N experts/models and make a prediction on if it rains or not tomorrow.
 - **Environment:** Observe the outcome.
-

Objective: follow the best expert advice (without knowing the best expert)

Prediction with Expert Advice

Aggregating weather prediction:

Initialization: N experts/models.

For each day $t = 1, \dots, T$:

- **Learner:** Combine predictions from N experts/models and make a prediction on if it rains or not tomorrow.
 - **Environment:** Observe the outcome.
-

Objective: follow the best expert advice (without knowing the best expert)

- How to combine the advices from N experts?

Aggregating weather prediction:

Initialization: N experts/models.

For each day $t = 1, \dots, T$:

- **Learner:** Combine predictions from N experts/models and make a prediction on if it rains or not tomorrow.
 - **Environment:** Observe the outcome.
-

Objective: follow the best expert advice (without knowing the best expert)

- How to combine the advices from N experts?
- How to update the belief on these experts? Exclude bad experts and find good experts?

Prediction with weighted majority algorithm (WMA):

Initialization: $w_1(i) = 1, \forall i \in [N]$.

For each day $t = 1, \dots, T$:

- **Learner:** if $\sum_{i \in S_t(A)} w_t(i) \geq \sum_{i \in S_t(B)} w_t(i)$ then choose A ; otherwise choose B .
 - **Environment:** Observe the outcome if it is A or B .
 - **Update:** $w_{t+1}(i) = (1 - \epsilon)w_t(i)$ for these experts that make mistake.
-

Intuition: trust experts with large weights

Prediction with weighted majority algorithm (WMA):

Initialization: $w_1(i) = 1, \forall i \in [N]$.

For each day $t = 1, \dots, T$:

- **Learner:** if $\sum_{i \in S_t(A)} w_t(i) \geq \sum_{i \in S_t(B)} w_t(i)$ then choose A ; otherwise choose B .
 - **Environment:** Observe the outcome if it is A or B .
 - **Update:** $w_{t+1}(i) = (1 - \epsilon)w_t(i)$ for these experts that make mistake.
-

Intuition: trust experts with large weights

- Combine the advices by “majority voting”.

Prediction with weighted majority algorithm (WMA):

Initialization: $w_1(i) = 1, \forall i \in [N]$.

For each day $t = 1, \dots, T$:

- **Learner:** if $\sum_{i \in S_t(A)} w_t(i) \geq \sum_{i \in S_t(B)} w_t(i)$ then choose A ; otherwise choose B .
 - **Environment:** Observe the outcome if it is A or B .
 - **Update:** $w_{t+1}(i) = (1 - \epsilon)w_t(i)$ for these experts that make mistake.
-

Intuition: trust experts with large weights

- Combine the advices by “majority voting”.
- Update the belief with multiplicative/exponential decay with learning rate ϵ .

Prediction with Expert Advice

Define M_T and $M_T(i)$ to be the total # of mistakes the WMA and the expert i make until T , respectively.

Theorem 1

For any expert $i \in [N]$, the weighted majority algorithm with the learning rate $(0 < \epsilon < 0.5)$ achieves

$$M_T < 2(1 + \epsilon)M_T(i) + \frac{2 \log N}{\epsilon}.$$

- # of mistakes the weighted majority algorithm is approximately twice # of mistakes the best expert makes.

Prediction with Expert Advice – Proof

A “potential/Lyapunov drift” style of analysis: define

$$\phi_t = \sum_{i \in [N]} w_t(i),$$

and study the drift

$$\phi_{t+1} - \phi_t \text{ or } \phi_{t+1}/\phi_t.$$

Prediction with Expert Advice – Proof

Expert problem:

Initialization: N experts/models.

For each day $t = 1, \dots, T$:

- **Learner:** Obtain predictions from N experts/models and choose an expert with probability p_t .
 - **Environment:** Observe the loss of each model $\ell_t \in [0, 1]^N$.
-

Objective: Find the best expert in hindsight, which is equivalent to minimize regret:

$$\mathcal{R}(T) := \sum_{t=1}^T \langle p_t, \ell_t \rangle - \sum_{t=1}^T \langle p, \ell_t \rangle = \sum_{t=1}^T \langle p_t, \ell_t \rangle - \sum_{t=1}^T \ell_t(i^*)$$

Hedge (Exponentially Weighted Average)

Initialization: $w_1(i) = 1, \forall i \in [N]$.

For each day $t = 1, \dots, T$:

- **Learner:** Compute the prob of choosing expert i :
 $p_t(i) = w_t(i) / \sum_i w_t(i)$.
 - **Environment:** Observe the loss of each model
 $\ell_t \in [0, 1]^N$.
 - **Loss update:** $w_{t+1} = w_t \cdot e^{-\eta \ell_t(i)}, \forall i \in [N]$.
-
- p_t is just the softmax function of accumulative loss.
 - A better expert with small accumulated loss $\sum_{s=1}^t \ell_s(i)$ will be selected with a high probability.
 - Can we find the best expert with Hedge? What is $\mathcal{R}(T)$?

Theorem 2

The regret of Hedge is

$$\begin{aligned}\mathcal{R}(T) &\leq \frac{\log N}{\eta} + \eta \sum_{t=1}^T \sum_{n=1}^N p_t(i) \ell_t^2(i) \\ &\leq \frac{\log N}{\eta} + T\eta.\end{aligned}$$

- Learning rate is important and let $\eta = O(\sqrt{\log N/T})$ provides $\mathcal{R}(T) = O(\sqrt{T \log N})$.
- $\lim_{T \rightarrow \infty} \mathcal{R}(T)/T = O(1/\sqrt{T}) \rightarrow 0$. Hedge can achieve the best expert asymptotically.

Expert problem: Regret of Hedge – proof

We study a “potential drift” proof with the potential function

$$\Phi_t = \sum_{i \in [N]} w_t(i).$$

Expert problem: Regret of Hedge – proof

Expert problem: Lower bound

We have derived the regret of $O(\sqrt{T \log N})$ for Hedge.

- Can we design even better policies/algorithms with the smaller regret?
- Is this improvable or optimal?

Expert problem: Lower bound

We have derived the regret of $O(\sqrt{T \log N})$ for Hedge.

- Can we design even better policies/algorithms with the smaller regret?
- Is this improvable or optimal?

To answer these questions, we need to study the lower bound.

- The lower bound indicates how hard the problem is.
- The lower bound is more about the problem itself and it is universal for any algorithms.
- Your policies/algorithms are good (optimal) if their performance are close to the lower bound.
- For Expert problem, what is the lower bound of regret?
Is it $\Omega(\sqrt{T \log N})$?

Expert problem: Lower bound

Theorem 3

For any algorithm, we have

$$\lim_{T, N \rightarrow \infty} \max_{\ell_1, \ell_2, \dots, \ell_T} \frac{\mathcal{R}(T)}{\sqrt{T \log N}} \geq \frac{1}{\sqrt{2}}.$$

- The asymptotically lower bound of expert problem is $O(\sqrt{T \log N})$.
- Hedge is asymptotically optimal!
- The proof can be found in that of Theorem 3.7 in the book “Prediction, Learning, and Games”³.

³A more accessible proof is Theorem 3 of Lecture 1 in a related course “Introduction to Online Learning” <https://haipeng-luo.net/courses/CSCI659>.